

Problemsetting

出题

出题前的准备

具备一定的水平

一方面，一个人自己出题，很难出难度大于自身水平的题目，一定的 OI 水平有助于想到更加优质的 idea 并想出优秀的做法；另一方面，OI 水平在一定程度上代表着 OI 资历，见识过更多的题目的选手也会对「好题」拥有自己的见解。

抱有认真负责的态度

出题是给别人做的，比起展示自己，更多是为了服务他人。算法竞赛是选手之间的竞赛，而不是出题人与做题人之间的较量。因此，出题不应以考倒选手为目标（当然，适当的防 AK 与良好的区分度也是非常重要的），而应当让选手能在比赛中有所收获。花费足够的时间精力去学习如何出题并认真负责地出题非常重要。

做好耗费大量时间的准备

如果想要认真地出题，就必然要花费大量的时间。如果不做好心理准备，可能导致比赛准备匆忙，质量不过关，也可能在事后由于没有将时间花费在学习上而懊悔。但出题也可以带来很多美好的回忆，如果真的对出题抱有兴趣，并做好了充分的心理准备，出题带来的收获也能够弥补那些花费的时间。

认真阅读本文的内容

本文从如何出题、如何把题出好两个方面对整个出题流程进行了介绍。对于想要出题的人来说，认真阅读本文一定能够受益匪浅。

题目内容

出一道题，idea，即题目本质的内容，是题目的灵魂，也是出题的第一步。

idea 的来源

1. 受到已有题目的启发（但不能照搬或无意义地加强，如：序列题目搬到仙人掌上）。
2. 受到学过的知识点的启发（但不能毫无联系地拼凑知识点）。
3. 从生活/游戏中受到启发（但注意不要把游戏出成大模拟）。
4. 不知道为什么，就是想到了一道题。

什么样的 idea 是不好的

关于原题

原题大致可分为完全一致、几乎一致和做法一致三种。

- 完全一致：使用一题的 AC 代码可以 AC 另一题。
- 几乎一致：由一题的 AC 代码改动至另一题的 AC 代码可以由一个不会该题的人完成。
- 做法一致：核心思路、做法一致，但代码实现上、不那么关键的细节上有差异。

这三种原题自下而上为包含关系。

以下情况不应出现：

1. 在明知有「几乎一致」的原题的情况下出原题。
2. 由于未使用搜索引擎查找导致自己不清楚有原题，从而出了「几乎一致」的原题。
3. 在「做法一致」的原题广为人知（如：NOIP、NOI 原题）时出原题。
4. 在带有选拔性的考试的非送分题中出现「做法一致」的原题。

以下情况最好不要出现：

1. 在明知有至少为「做法一致」的原题的情况下出原题。
2. 由于未使用搜索引擎查找导致自己不清楚有原题，从而出了「做法一致」的原题。
3. 在任何情况下出「几乎一致」的原题。

可以放宽要求的例外情况：

1. 校内模拟赛。

2. 以专题训练为目的的模拟赛。
3. 难度较低的比赛，或是定位为送分题的题目。

关于毒瘤题

「毒瘤题」是一个非常模糊而主观的观念，在这只是引用一些前人关于此的探讨，加以自己的一些理解。这个话题是非常开放的，欢迎大家来发表自己的观点。

一道好题不应该是两道题拼在一起，一道好题会有自己的 idea——而它应该不加以过多包装地突出这个 idea。

一道好题应该新颖。真正的好题，应该是能让人脑洞出新的好题的好题。

——vfk《UOJ 精神之源流》

例子：[「XR-1」柯南家族](#)，做法的前后两部分完全割裂，前半部分为 [「模板」树上后缀排序](#)，后半部分是经典树上问题。就算是随意输入树的点权，依然可以做第二部分，前后部分没有联系。

一类 OI 题以数学为主，无论是题目描述还是做法都是数学题的特征，并且解法中不含算法相关的知识点，这类 OI 题目统称为纯数学题。

——王天懿《论偏题的危害》

经典例子：[NOIP2017 小凯的疑惑](#)

OI 中的数学题与其它数学题的区别，也是体现 OI 本质的一个特点，是 OI 中的数学题往往重点不在答案是什么，而在如何加快答案的计算。如果一道题考察的重点是「怎么算」而非「怎么快速计算」，这样的数学题一般都不适合出在 OI 中的。

一部分偏题中牵涉到了大学物理的内容，导致选手在面对这些从未接触过物理知识点时变得不知所措，造成了知识上的隔膜。

——王天懿《论偏题的危害》

经典例子：[「清华集训 2015」多边形下海](#)

不止是物理，OI 题目中不应过多涉及到其它学科的知识，如果涉及应当给予详细的解释，不应使其它学科的知识作为解题的重大障碍。

一道好题无论难度如何，都应该具有自己的思维难度，需要选手去思考并发现一些性质。

一道好题的代码可以长，但一定不是通过强行嵌套或者增加条件而让代码变长，而是长得自然，让人感觉这个题的代码就应该是这么长。

——王天懿《论偏题的危害》

经典例子：[「SDOI2010」猪国杀](#)，[「集训队互测 2015」未来程序·改](#)

在一般的 OI 比赛中，思维难度应占主要部分。当然，如 THUWC/THUSC 的 Day 2+ 那样的工程题也有其存在的道理——毕竟体验营的目的除了考察选手的算法设计能力，还有和大学学习对接的工程代码以及文档学习能力。但在一般的 OI 比赛中，考察更多的应当还是算法设计与思维能力。

题面

使用 LaTeX 书写公式

网上有很多 LaTeX 的教程，如：

- [LaTeX 入门](#)
- [LaTeX 数学公式大全](#)
- [LaTeX 各种命令、符号](#)

使用时请注意 [LaTeX 公式的格式要求](#)。

题目背景

题目背景最好尽量简短。在题目背景较长时，应当与题目描述分开。

需要绝对避免题目背景严重影响题意的理解。

必要时，可以提供与背景结合的题目描述与简洁的题目描述两个版本。

题目描述

简而言之，题目描述需要清晰易懂。

题面中的每个可能不被理解的定义都应得到解释，不应凭空冒出未加定义的概念。例如：在 [CF1172D Nauuo and Portals](#) 中，你必须在题面中解释什么是「传送门」。

题面中涉及到的每个概念应当使用单一的词汇来描述。例如：不应一会儿说「费用」，一会儿说「代价」。

不应不加说明地使用与原义、常见义不同的词汇。例如：不应不加说明地用「路径」代指一条边。

你需要保证你的题面不会自相矛盾。例如：在 [CF1173A Nauuo and Votes](#) 中，没有把 "?" 作为一种 "result"，是因为 "?" 的含义是 "there are more than one possible results"。

你需要保证你的题面不能被错误理解而自圆其说，即使这种理解是反常识、没有人会这么去想的。例如：在 [CF1172D Nauuo and Portals](#) 中，之所以要繁琐地定义 "walk into" 并与 "teleport" 区分，是为了防止这种理解：通过传送门可以到另一个传送门，而到了传送门会传送，因此会反复横跳。

顺着读题目描述应当能看懂每一句话，并理解题目的任务与要求。至少在紧接着的下一段话中疑惑能够得到解释，而不是需要在若干段后才能得到解释，或者要看了输入输出格式才能明白题意，甚至需要根据样例来猜题意。例如：在「[GuOJ Round #1](#)」[琪露诺的冰雪宴会](#) 中，在输出格式才第一次出现了题目的目标「雾之湖最终能接收到的最大水量」，再加上「灵梦当然能很快算出来清理完全部小溪的总费用是多少」这句带有误解性质的话，更容易使人读错题意，这是不可取的，应当在题目描述中就对题目的目标进行说明。（在这个例子中还存在题目背景严重影响题意理解的问题。）相同的错误还出现在 [CF1423\(4\)N Bubblesquare Tokens](#) 中，在输出格式才第一次出现了题目的目标 "friend pairs and number of tokens each of them gets on behalf of their friendship"。

输入输出格式

输入输出格式清晰 **完整** 即可，没有死板的要求，个人建议参照 CF 的题目来写输入输出格式，具体可以参考[CF 出题人须知](#)。

为了方便选手做题，输入输出格式中最好说明每个变量的具体含义，除非变量的意义非常长，没法一句话说明清楚（这时可以说「意义见题目描述」）。

需要特别注意的是，如果输出中含有小数，请尽量使用 [SPJ](#) 来对误差的大小进行限制，而非要求「保留 x 位小数」。

「保留 x 位小数」对精度的要求可能是无限的。例如：要求保留三位小数，实际答案为，此时只要有任意大小的误差导致计算出的答案小于，即使计算出的答案是 也会输出错误的答案。

如果无法使用 SPJ，请保证对精度的要求是有限的，例如：请输出答案四舍五入后保留小数点后三位的结果。令标准答案为，数据保证对于任意满足的，四舍五入后结果与 四舍五入后相同。

可以参考的一些句子：

1 输入的第一行包含三个正整数 n, m, k ($1 \leq n, m \leq 2 \cdot 10^5, 1 \leq k \leq 100$) — n 表示数列的长度， m 表示操作个数， k 的意义见题目描述。

1 输入的第二行包含 n 个非负整数 $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^9$) — 题目给出的数列。

1 接下来的 m 行中的第 i 行包含两个正整数 l_i 和 r_i ($1 \leq l_i \leq r_i \leq n$)，表示第 i 次操作在区间 $[l_i, r_i]$ 上进行。

1 接下来的 $n-1$ 行，每行包含两个正整数 u 和 v ($1 \leq u, v \leq n$)，表示 u 和 v 之间由一条边相连。

2

3 数据保证给出的边能构成一棵树。

1 输入的唯一一行包含一个由小写英文字母构成的非空字符串，其长度不超过 10^6 。

1 输入的第二行包含一个小数点后不超过三位的实数 x ($-10^6 \leq x \leq 10^6$)，意义见题目描述。

1 输出包含一个实数，当你的输出与标准答案之间的绝对误差或相对误差小于 10^{-6} 时视作正确。

1 输出的第二行包含 n 个正整数，表示你构造的一组方案 — 其中第 i 个数表示你打出的第 i 张牌的编号。

2

3 如果有多组合法的答案，可以任意输出其中一组。

▼ 在选手代码内由随机数生成器生成输入数据

有的题目会因为输入数据过大，为了防止读入用时过长，而要求选手在代码内通过给定的数据生成器生成数据，代替通过标准输入或文件输入来读入数据。

采用这种做法需要谨慎考虑，因为它有很多缺点：

- 可能引入了正解所不需要的数据随机性，或者使得构造数据变得困难
- 可能增大了理解输入格式的难度
- 如果随机数生成器封装的不好，可能理解数据生成器本身的使用方法就有难度
- 如果选手没有使用出题者推荐的语言，可能需要自己写一个数据生成器

采用这种做法一般是为了防止读入数据用时过长，所以一个可能的替代方案是下发一个性能足够好的 [读入、输出优化](#) 模板，以尽量保证所有人的读入用时一致，这样的话即使读入用时很久也不会影响不同选手用时的差异。另一个解决方案是将题目包装成函数调用式（而非 IO 式）交互题，即使算法

过程中没有交互，交互题也可以起到统一读入时的作用，IOI 就采用了所有题目都是交互题的方案。但是，这两种方案都对选手使用的语言有限制，需要出题者手动支持每种允许选手使用的语言。

回到问题的本源，还可以考虑一下过大的输入数据是否是必要的，有没有可能使用较小的输入数据达到目的，以及比正解复杂度稍劣的做法是否有卡掉的必要。

数据范围

按照 CF 的要求，数据范围要写在输入格式里，但在国内，数据范围往往是写在题目的最后的。

数据范围中最容易犯的错误就是不完整。输入中的每一个数、每一个字符串都应该有清晰的界定。在上文所给出的输入输出格式示例中就有一些数据范围的正确写法。

数据范围的常见遗漏：

1. 「整数」中的「整」。
2. 题面中只说了是「整数」没说是「正整数」，并且数据范围中只有上限没有下限。
3. 字符串没说字符集。
4. 实数没说小数点后位数。
5. 某些变量没有给范围。

你需要保证标程可以通过满足题面所述数据范围的 **任何一组数据**。

▼ 关于「保证数据随机生成」

有的题目中会「保证数据随机生成」，很多时候这样的限制并不是最优的解决方案，因为「随机生成」对数据的限制并不明确，会给判断具体数据范围、提供 hack 数据带来困难。

一般来说，「保证数据随机生成」可以换成解法所需要的数据性质。例如，随机生成一棵树往往可以换成限制树的高度。

如果一定要保证数据随机生成，应当指定随机生成的具体操作。例如，生成一棵树是随机选择父亲节点还是随机生成 Prüfer 序列。

需要注意的是，非确定性算法和依赖于数据随机性的算法是不同的。前者可以对于任意数据都有很高的概率得到正解，而后者是对于大部分的数据能得到正解，对于某些特定的数据则不可能得到正解。

样例

样例应当有一定的强度，能够查出一些简单的错误。读错题意的人应当能够通过样例发现自己读错了题意。

有多种操作的题，每种操作都应在样例中出现。

有多种输出的题（如 [CF1173A Nauuo and Votes](#)），每种输出都应在样例中出现。例外：实际上不可能无解，但要求判断是否有解的题目。

样例解释

题目描述越复杂、越不易理解就越应当有详细的样例解释。

题目难度越简单就越应当有详细的样例解释。

详细的样例解释可以选择配上图片。

较大的样例可以没有样例解释。

为了照顾色觉障碍者，最好不要使颜色成为理解样例解释所必备的。可以用彩色图片来美化样例解释，但如果一定要用颜色传递一些必要的信息，最好不要同时出现红黄或者红绿。

时限、空间限制与部分分

时限与空间限制的目的是卡掉复杂度错误的做法。（当然，也是为了防止评测用时过长，如：只对交互次数有限制而对时间复杂度没有限制的交互题也有时间限制。）

因此，原则上时间限制应当选取不使错误做法通过的尽量大的值。

一般地，时限应满足以下要求：

1. 至少为 std 在最坏情况下用时的两倍。
2. 如果比赛允许使用 Java，应使 Java 能够通过。
3. 不应使错误做法通过（实在卡不掉、想放某种错解过除外）。

为了更好地在放大常数做法过的同时卡掉错解，一般可以采用同时增大数据范围和时限的方法。但要注意，有时正解（由于缓存等玄学问题）会在数据范围增大时有极大的常数增加，此时增大数据范围不一定能够增大正解与错解之间用时的差距。

在有部分分的赛制中，还可以通过设置有梯度的数据、数据范围稍小的数据来使较为优秀的错解和大常数正解不能通过，同时使其获得较高的部分分。

需要注意的是，在数据范围小于 时，应当考虑是否能使用 [指令集](#) 通过。

一般情况下空间限制应当设置的足够大，除非空间复杂度更优的做法的确十分巧妙，值得卡掉空间复杂度大的做法。这种情况下可以考虑设置空间限制较松的部分分。值得注意的是，如果不想卡掉空间消耗较大的做法，数据结构题一般需要设置较大的空间限制。

一道好题应该具有它的选拔性质，具有足够的区分度。应该至少 4 档部分分，让新手可以拿到分，让高手能够展示自己的实力。

——vfk《UOJ 精神之源流》

部分分一般分为较小数据范围与特殊性质两种。

较小数据范围一般要设置多档，即使你想不到某种复杂度的做法，也可以考虑给这种复杂度一档分。一般来说，为了避免卡常，可以设置一档极限数据除以二的部分分。

「数据有梯度」最好用多档部分分替代。

特殊性质部分分的设置要依具体题目而定。理想的特殊性质部分分应当是能够引导选手思考正解的。与较小数据范围部分分不同，在你不会针对某种特殊性质的做法时，最好不要给这种特殊性质一档分。例如：[「CTS2019」随机立方体](#) 的 这档部分分在讲题时就被很多人吐槽，称这档部分分妨碍了思考正解。

如果题目给分方式与默认方式不同（如：在一般的 OI 赛制比赛中绑 subtask 测试），一定要在题面中说明。

不推荐使用「百分之 XX 的数据满足 XX」的说法，尤其是数据范围有多个变量时。例如，「的数据满足」和「的数据满足」可能描述了 的数据的性质，也可能只描述了 数据的性质。一般来说，subtask 或数据范围表格是更好的选择。

造数据

数据生成是出题过程中必要的一步，也是对拍时所必需的，掌握一些生成数据的技巧，就能使造数据的过程更加轻松，造出来的数据强度更高。

生成随机数据

生成随机数

请参考 [随机函数](#) 页面。

需要特别提醒的是，在生成值域比随机函数返回值更大的数时，请 **不要** 使用 `rand() * rand()` 之类的写法，这样的写法生成的随机数非常不均匀。

另外，出题时推荐使用 [testlib](#) 来造数据，可以保证在不同平台上同一个种子生成的随机数相同，并且种子会依据命令行参数自动生成。

生成随机排列

可以使用 STL 中的 `std::shuffle` 函数，形如 `std::shuffle(a, a + n, rng)`，这里 `rng` 是一个随机数生成器，比如 `std::mt19937 rng(std::chrono::steady_clock::now().time_since_epoch().count())`。

请 **不要** 使用 `std::random_shuffle`，它在 C++14 中弃用，C++17 中被移除。

生成随机区间

常见错误方法：在 中随机生成左端点，再在 中随机生成右端点。这样的话生成的区间会比较靠右。

较为正确的方法（推荐做法）：在 中随机生成两个数，取较小的作为左端点，较大的作为右端点。

真正均匀随机的方法：在 中生成一个随机数，若，再在 中生成一个随机数，区间为；否则按「较为正确的方法」生成。

生成随机树

常用方法是为 的每个节点 从 中随机选择一个父亲。这样的话生成的树不是均匀随机的，期望高度为 。

还有一种随机方法：从 中随机选择 的父亲。若 和 设置得当，可以造出强度较高的树。

真正均匀随机的方法是利用 [Prüfer 序列](#)，先生成一个随机 Prüfer 序列，再通过序列生成树。这样的话，树的期望高度为 。

除此之外，可以随机一个排列来给节点重编号/打乱边的顺序。

构造数据

区间相关的题目

常用构造：长度特别小（特殊地，全部为单点）、长度特别大（特殊地，全部为整个序列）。

需要分解因数的题目

可重质因数个数尽量多： 的幂。

去重后质因数个数尽量多：最小的若干个质数相乘。

约数尽量多：可以参考 OEIS 上的 [A002182](#) 数列。

树上问题

常用构造：

- 链
- 菊花
- 完全二叉树
- 将完全二叉树的每个节点替换为一条长为 的链
- 菊花上挂一条链
- 链上挂一些单点
- 一棵高度为 且 的树的根节点有两个儿子，左子树是一条长为 的链，右子树是一棵高度为 的这样的树。

如果不是在考场上，还可以使用 [Tree-Generator](#) 来生成各种各样的树。

批量生成数据

笔者推荐使用命令行参数 + bat/sh 的方法。

例如：

gen.cpp:

```
1 #include "testlib.h"
2
3 using namespace std;
4
5 int n, m, k;
6 vector<int> p;
7
8 int main(int argc, char* argv[]) {
9     registerGen(argc, argv, 1);
10
11     int i;
12
13     n = atoi(argv[1]);
14     m = atoi(argv[2]);
15     k = rnd.next(1, n);
16
17     for (i = 1; i <= n; ++i) p.push_back(i);
18
19     shuffle(p.begin(), p.end());
20     // 使用 rnd.next() 进行 shuffle
21
22     printf("%d %d %d\n", n, m, k);
23     for (i = 0; i < n; ++i) {
24         printf("%d%c", p[i], " \n"[i == n - 1]);
25         // 把字符串当作数组用，中间空格，末尾换行，是一个造数据时常用的技巧
26     }
27
28     return 0;
29 }
```

gen_scripts.bat:

```
1 gen 10 10 > 1.in
2 gen 1 1 > 2.in
3 gen 100 200 > 3.in
4 gen 2000 1000 > 4.in
5 gen 100000 100000 > 5.in
```

这样做的好处是，对于不同的数据只需要写一个 generator，并且可以方便地修改某个测试点的参数。

造数据的要求

数据应当包含各个参数的最小值和最大值。

数据应当包含各种边角情况。

在使用 `subtask` 时，数据（包括输入、输出）最好覆盖到值域中的各个范围，而不是只有数据范围的最大值。

为了防止针对特殊构造的特判过掉，可以将不同的构造结合在一个测试点中，或者数据的大部分是构造，掺杂小部分的随机。

数据中应当包含各种各样的构造，即使你不知道什么错解会挂在这组构造上。（在按测试点给分的赛制中需要酌情处理。）

当然，如果你已知一个（正常人能想的到、写的出的）正确性有问题的错解，要尽量卡掉它。

需要特别提醒的是，如果有整型溢出的可能，一定要卡掉溢出的做法。在有部分分的赛制中，不应使不开 `long long` 的人得到和暴力一样甚至更低的分数。

如果有 `pretests`，`pretests` 应尽量强，（同时尽量少）。换言之，你需要在 `pretests` 中（用尽量少的数据组数）包含该题的所有已知叉点。

如果你希望出现少量而非没有 `FST`，仍然应当保证 `pretests` 的强度，因为实际比赛中很可能出现你意想不到的错误，导致远远高出预期的 `FST` 数量。

数据的格式

这里提供一些通常情况下输入数据的格式要求，可作为一般情况下的参考：

1. 使用测试环境下的换行格式。
2. 文件最后一行的末尾有换行符，即整个文件的最后一个字符需要是 `\n`。
3. 任何一行的开头和末尾都没有空白字符。
4. 连续的空格不超过 1 个。

在 Windows 环境下生成的数据，其换行格式通常为 `\r\n`，而主流测评系统均在 Linux 环境下运行，其换行格式为 `\n`。若在 Linux 环境下读入 Windows 格式的换行数据，可能会导致读入字符串时换行处理异常，进而导致不同环境下程序运行结果不同；若在 Linux 环境下比较 Linux 环境下生成的输出和 Windows 环境下生成的标准输出，可能由于换行格式不同而导致比较存在差异。为了保持程序行为一致，所有数据的换行格式必须转换为程序运行环境下的换行格式。

一般可以通过如下方式生成 Linux 格式换行的数据：

1. 直接使用 Linux 环境生成数据。
2. 通过 [dos2unix](#) 工具对输入输出文件进行转换，此工具包含于 Cygwin, MinGW 等工具链中。
3. 使用二进制方式打开输出文件，并且使用 `\n` 换行格式。
4. 参考 [此页面](#) 中 `dos2unix.cpp` 代码自行编写工具。

Special Judge

SPJ 编写教程

输出方案题和输出浮点数题是两种较为常见的需要使用 SPJ 的题型，其它题目视情况也需要使用 SPJ。在 CF 上，所有题目都必须使用基于 `testlib` 的 checker，例如：题目要求输出若干个整数时，使用 `testlib` 自带的 `ncmp` checker，选手可以任意输出空白字符（既可以空格也可以换行）。

checker 一般使用 `testlib` 编写。由于 checker 要应对各种各样的不合法输出，需要极强的鲁棒性，不使用 `testlib` 是很难写好 checker 的。

编写 checker 需要注意以下两点：

1. 你需要应对各种不合法的输出，因此，请检查读入的每个变量是否在合法范围中（`readInt(minvalue, maxvalue)`）。例如：读入一个在 `check` 过程中会作为数组下标的变量时必须检查其范围，否则可能引发数组越界，有时这会导致 RE，有时则可能判为 AC。
2. 原则上 checker 中不应检查空白字符（即，不应使用 `readSpace()`、`readEoln()`、`readEof()`），值得一提的是，`testlib` 会自动检查是否有多余的输出）。

题解

题解的目标是让预计会来参加比赛的人都能看懂。所以官方题解详细程度的要求会比一般的题解高。

关于部分分

在有部分分的题目中，题解里可以考虑写一写部分分的做法。

关于知识点

解题中用到的知识点应当明确指出。对于一些难度和题目难度相当的知识点，最好给出学习该知识点的资料（比如一篇博客的地址）。

关于定义

题解中不要凭空冒出来一些概念。

例如：dp 的题解要解释清楚状态的定义。

关于细节

具体的实现细节如果比较巧妙最好写出来，否则的话「详见代码」也是可以的。如果「详见代码」的话，最好在代码中加上一定的注释。

标程

标程中最好去掉冗余部分。比如，有的题解中保留了完整的 `define` 模板（为了提高做题速度，包含大量 `define` 与常用函数，常用于 CF 等在线比赛），并且其中很大一部分都没有用到，这是不好的。

如果涉及到一些题解中没有详细实现的实现细节，最好加上适量的注释。

比赛

比赛通知中的题目难度需真实

Remember that authors tend to underestimate the difficulty of their problems.

——Codeforces PROPOSE A PROBLEM 页面的提醒

出题人很可能错误估计题目的难度，因此，如果要在比赛通知中写上比赛难度，需要谨慎考虑，最好提前请人来验题并进行评估。

题目难度的分配

在美国内 OI 的模拟赛中，往往是三道题的整体难度与比赛难度相当即可。

在类 CF/ATC 这种线上赛的比赛中，需要尽量保证难度的递增（虽然由于对难度的误估很多时候都不能真正做到），并且尽量避免出现大的 difficulty gap。可以通过把一题分为难易两题（两个 subtask）来减少 difficulty gap，但是分 subtask 需要谨慎考虑，也有很多人不喜欢 CF 赛制中的 subtask（[Are subtasks evil?](#)），原因包括但不限于：

- 由于赛制原因，可能先做 easy version 再做 hard version 罚时更少而总分更高
- subtask 的赋分往往与题目难度不成正比
- 很多时候 easy version 的题目并不是一道合格的题目（不有趣）
- 很多时候 easy version 的解法对于思考 hard version 的正解没有帮助

题目知识点的分配

一场比赛应尽量涵盖较广的知识点（专题训练赛当然除外）。

经典反例：涵盖了动态规划、期望、组合计数、容斥原理、多项式等多种知识点的 CTS2019。

我要从五道题里选六道，我也很无奈啊。

——CTS2019 组题人给出的理由，没有收到足够多的题目投稿

出题平台

Polygon

Polygon 是一个功能非常强大的多人合作出题平台，可以作为在任何网站（使用 package 功能导出到不支持 Polygon 的网站）多人合作出题的首选方案，单人出题（尤其是在不同设备上出题）时也是很不错的选择，使用方法参见 [Polygon 简介](#)。

Codeforces

Codeforces 是全球最著名的算法竞赛网站之一，题目质量较高，非常适合有一定出题经验并且想进一步提升出题水平、想要出一套高质量题目的出题人。不足之处是审核速度较慢（一般要几个月），但你也可以在审核期间就开始题目的准备（虽然有题目被否掉导致准备白费了的风险）。

出题资格

- 蓝名且参加过至少 25 场 rated 比赛；
- 紫名且参加过至少 15 场 rated 比赛；
- 橙名且参加过至少 5 场 rated 比赛；
- 红名或黑红名。

提交比赛申请

有了出题资格后，在侧边栏可以看到 [Propose a contest/problems](#) 按钮。

点进去之后，先写一份 contest proposal（在 PROPOSE A CONTEST 里写），然后再写 problem proposal 并添加进比赛里。

题目决定好之后，就可以将 contest proposal open to review（提交审核）了。

在 Polygon 上准备题目

参考 [Polygon 简介](#)。

与管理之间的联系

与管理联系有两个作用：

1. 加快审核速度。
2. 进入准备阶段后管理会提供建议和帮助。

正规的联系方式是在 proposal system 中以 proposal 的形式提交申请，管理开始审核之后以 comment 的形式在 proposal 的下方进行讨论。

实际上，如果 proposal 长时间没有过审，可以考虑私信联系管理（其实 CF 上写了 "Don't send private messages or emails to coordinators"，但 300iq 在 [评论](#) 中表示可以私信他）。

Comet OJ

[Comet OJ 链接](#)

已经不再活跃（截至 2021 年 11 月，最后一场比赛是 2020 年 1 月的）。

出题申请：https://info.cometoj.com/contests/Questionnaire_IssuerInfo/

CodeChef

印度的算法竞赛平台，有三种赛制：10 天且带 challenge 的 Long Challenge，2.5h 类 ICPC 的 Cook-Off，3h 类 IOI 的 LunchTime。

出题 FAQ：<https://www.codechef.com/wiki/faq-problem-setters>

出题指南：<https://www.codechef.com/problemsetting>

AtCoder

日本的算法竞赛平台，出题联系方式：contest@atcoder.jp。

UOJ & LOJ

比赛不多的国内 OJ。

洛谷

个人公开赛

在「我的题库」中出题并提交比赛申请。

团队公开赛

在团队页面中出题并提交比赛申请。

参考资料

1. [vfk《UOJ 精神之源流》](#)
2. [王天懿《论偏题的危害》](#)
3. [CF 出题人须知（国内可访问的图片版）](#)
4. [CF 出题人的自我修养](#)

本文由作者本人自 [ouuan 的出题规范](#) 搬运而来并有所修改、补充。

本页面最近更新：2025/3/16 17:12:58, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者：[ouuan](#), [StudyingFather](#), [ChungZH](#), [Cryflmind](#), [Henry-ZHR](#), [xyf00Z](#), [abc1763613206](#), [CCXXI](#), [Enter-tainer](#), [HeRaNO](#), [Ir1d](#), [jifbt](#), [mgt](#), [NachtgeistW](#), [shuzhouliu](#), [sy201901y](#), [TianyiQ](#), [uniHk](#), [Xeonacid](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用